

Proxy-Informed Gaussian Process Priors for Bayesian Optimisation of Very Expensive Objectives

James Avery

Department of Computer Science, University of Copenhagen

avery@diku.dk

July 2026 — working draft

Abstract

Bayesian optimisation (BO) of very expensive objective functions — e.g. coupled-cluster electronic structure energies that take months per evaluation — typically starts from an uninformative Gaussian process (GP) prior and must spend a large fraction of its small evaluation budget rediscovering the gross structure of the objective. In many such applications a fast surrogate of the objective exists (harmonic approximations in microseconds, semi-empirical methods in seconds) together with a *known pointwise bound* on its error: the true objective deviates from the proxy by at most a known percentage, even though the locations of their optima may differ. We show how to encode this knowledge exactly where it belongs in the BO machinery: the GP prior mean is set to the proxy, and the prior covariance is scaled heteroscedastically so that the prior standard deviation at every point is the promised deviation band. The bound is treated as a hard upper limit on the prior amplitude — a single trainable scale factor may tighten the band as data accumulates but is constrained never to exceed it. The construction composes with arbitrary stationary correlation kernels and survives the data renormalisation used in practical BO implementations. We document the implementation in the open-source package `veropt` down to the level of the code that realises each equation, give a user guide with a complete, runnable example, and benchmark the method: on the six-dimensional Hartmann function with a proxy distorted by at most 1%, the proxy-informed prior comes within 0.2% of the global optimum after a single Bayesian batch (12 evaluations) in every repetition, whereas a flat prior is still 18% short of the optimum when the budget of 24 evaluations is exhausted.

1 Introduction

Bayesian optimisation is the method of choice for optimising black-box functions whose evaluations are so expensive that only a small number — often around one hundred — can be afforded (Shahriari et al., 2016; Frazier, 2018). A Gaussian process surrogate is fitted to the evaluations made so far, and an acquisition function balances exploration against exploitation to select the next batch of points.

The standard GP prior is maximally uninformative: a constant mean and a stationary kernel with a single global amplitude. Every evaluation must then contribute to learning both the gross structure of the objective *and* the fine structure near the optimum. For the most expensive objectives this is wasteful, because the gross structure is often already known. Our motivating example is molecular electronic structure: a CCSD(T)-level energy surface may cost months of computation per point, while a harmonic approximation to the same surface is available in microseconds and semi-empirical electronic structure methods in seconds. Crucially, these cheap surrogates frequently come with *quantitative* error guarantees of the form “the true energy is within 1% of the approximation, everywhere”. The minima of the cheap and the expensive

surface need not coincide — otherwise the problem would be solved — but pointwise, the expensive function is trapped in a known band around the cheap one.

This paper describes a simple, well-founded way to inject exactly this kind of knowledge into BO: build the GP prior from the proxy. The prior mean is the proxy itself, and the prior covariance is rescaled so that the prior standard deviation at each point equals the promised band. The result is that the surrogate starts out already knowing everything the proxy knows, and the expensive evaluations are spent exclusively on learning the *residual* between truth and proxy — a function whose amplitude is bounded by the guarantee. Related ideas appear in multi-fidelity BO (Kennedy and O’Hagan, 2000; Forrester et al., 2007), where cheap and expensive evaluations are modelled jointly, and more broadly in grey-box BO (Astudillo and Frazier, 2021), which exploits known structure of the objective. The variant presented here is deliberately minimal: it requires no evaluations of the expensive function to fit the relationship between fidelities — the auto-regressive coefficient and discrepancy process of Kennedy and O’Hagan (2000) are replaced by an identity coupling and an amplitude bound, both supplied a priori by the user’s guarantee. This matters precisely in the regime we target: with months per evaluation, there is no budget to learn the cross-fidelity relationship empirically.

The paper is organised so that a reader can go from the mathematics to a working optimisation of their own problem. Section 2 develops the construction and the two subtleties that make it work in practice (kernel-diagonal calibration and behaviour under data whitening). Section 3 documents how the construction is realised in the open-source package `veropt` (Stoustrup et al., 2026), quoting the code that implements each equation and describing every change made to the existing optimiser core. Section 4 is a user guide: the contract a proxy must satisfy, the configuration options, a complete runnable example, and a recipe for adapting it to a new problem. Section 5 benchmarks the method.

2 Method

2.1 Setting

Let $f: X \subset \mathbb{R}^d \rightarrow \mathbb{R}$ be the expensive objective, to be maximised under a budget of $N \sim 10^2$ evaluations. (`veropt` always maximises; an objective to be minimised, such as an energy, is negated — nothing in the construction depends on the sign convention.) Let $g: X \rightarrow \mathbb{R}$ be a fast proxy (μ s-s per evaluation), and assume the pointwise guarantee

$$|f(x) - g(x)| \leq \varepsilon(x) \quad \text{for all } x \in X, \quad (1)$$

with ε known. The two cases of practical interest are the *relative* bound $\varepsilon(x) = r |g(x)|$ (“ f is within 1% of g ”, $r = 0.01$) and the *absolute* bound $\varepsilon(x) = \varepsilon_0$ (“ f is within 1 kcal/mol of g ”). Nothing is assumed about the locations of the optima of f and g .

2.2 The proxy-informed prior

We place on f the Gaussian process prior

$$f \sim \mathcal{GP}(m, k), \quad m(x) = g(x), \quad k(x, x') = c^2 \sigma(x) \sigma(x') \rho(x, x'), \quad (2)$$

where ρ is a correlation kernel ($\rho(x, x) = 1$) and

$$\sigma(x) = \frac{\varepsilon(x)}{\kappa}, \quad (3)$$

with $\kappa > 0$ a confidence parameter discussed below, and $c \in (0, 1]$ a global amplitude factor discussed in Section 2.3. The kernel in (2) is the standard non-stationary scaling of a correlation

kernel by an outer product of positive functions, and is a valid covariance: for any finite set of points $\{x_i\}$ the Gram matrix factors as $K = DRD$ with $D = \text{diag}(c\sigma(x_i))$ and R the (positive semi-definite) correlation Gram matrix, and congruence by a diagonal matrix preserves positive semi-definiteness.

In practice ρ is obtained from an arbitrary base kernel k_b by normalising with its diagonal,

$$\rho(x, x') = \frac{k_b(x, x')}{\sqrt{k_b(x, x) k_b(x', x')}}, \quad (4)$$

so that the construction stays calibrated even for base kernels with $k_b(x, x) \neq 1$ — kernel sums and spectral mixtures being the common offenders: a sum of two Matérn kernels has $k_b(x, x) = 2$ and would otherwise silently inflate the band by $\sqrt{2}$ past the promised bound. Because $\rho(x, x) = 1$ exactly by (4), the prior variance at any point is $c^2\sigma(x)^2$ by construction — there is no additional global outputscale to interfere with the calibration. The hyperparameters of k_b (e.g. ARD lengthscales) remain free and are trained as usual; they control the *correlation structure* of the residual, while its *amplitude* is fixed by the guarantee.

Equation (2) is most easily read through the residual $h = f - g$: the prior states $h \sim \mathcal{GP}(0, k)$ with pointwise standard deviation $c\sigma(x)$, i.e. the model believes the truth deviates from the proxy by a smooth function whose typical size at x is a known fraction of the band $\varepsilon(x)$. With $c = 1$ and $\kappa = 2$ the prior places $\sim 95\%$ of its mass inside the band (1) at every point; $\kappa = 3$ gives $\sim 99.7\%$. The bound is thus encoded *softly* — a GP cannot represent a hard truncation — and κ expresses how literally the guarantee is taken. We default to $\kappa = 2$.

A worked number makes the calibration concrete. Suppose the objective is a negated total energy and the proxy predicts $g(x) = -50.0$ energy units at some point x , with a relative guarantee of 1% ($r = 0.01$). Then $\varepsilon(x) = 0.01 \times 50.0 = 0.5$, and with $\kappa = 2$ the prior at x is $f(x) \sim \mathcal{N}(-50.0, 0.25^2)$: its central $\pm 2\sigma$ interval is $[-50.5, -49.5]$ — exactly the promised band. Far from any expensive evaluation the posterior returns to this prior, so the surrogate is never more ignorant than the guarantee allows, anywhere. If training later tightens the amplitude to $c = 0.8$ (Section 2.3), the band at x narrows to ± 0.4 .

2.3 The bound as a hard cap on a trained amplitude

The guarantee (1) is an *upper* bound: the proxy may well be better than its worst case. We therefore make the global amplitude factor c in (2) a hyperparameter, trained alongside the kernel lengthscales by type-II maximum likelihood, but constrained to

$$c \in (c_{\min}, 1], \quad (5)$$

implemented by a sigmoidal reparametrisation (a `gpytorch` (Gardner et al., 2018) interval constraint). The cap at 1 guarantees that no amount of hyperparameter optimisation can inflate the prior beyond the promised band, while the marginal likelihood is free to *tighten* the band as the data reveal that the proxy is more accurate than worst-case — which immediately sharpens the posterior everywhere, not only near evaluations. The floor $c_{\min}$ (default 0.1) prevents overconfident collapse from the handful of points available early on. Setting c constant at 1 recovers the literal worst-case prior.

One practical guard is worth stating here because it shapes the user-facing options: under a relative bound $\sigma(x) \rightarrow 0$ wherever $g(x) \rightarrow 0$, which pins the GP to the proxy and can destabilise Cholesky factorisations. A small floor $\varepsilon(x) \leftarrow \max(\varepsilon(x), \varepsilon_{\text{floor}})$ restores numerical safety; the user chooses $\varepsilon_{\text{floor}}$ in the objective’s own units.

2.4 Interaction with data normalisation

Practical BO implementations whiten their data, and this is where a naive implementation of (2) silently breaks. In `veropt`, inputs and observed objective values are affinely normalised to

zero mean and unit variance per dimension,

$$T_x(x) = (x - \mu_x) \oslash s_x, \quad T_y(y) = (y - \mu_y)/s_y, \quad (6)$$

(with $\oslash$ elementwise division), and — crucially — the normalisers are *refitted* as new evaluations arrive, by default at every step in the multi-objective case. The GP model lives entirely in whitened coordinates; the proxy is defined in physical units. The prior mean must therefore be evaluated as

$$\tilde{m}(\tilde{x}) = T_y(g(T_x^{-1}(\tilde{x}))) = \frac{g(T_x^{-1}(\tilde{x})) - \mu_y}{s_y}, \quad (7)$$

where tildes denote whitened quantities. The band, being the standard deviation of a *deviation* rather than a value, transforms through the scale of T_y only:

$$\tilde{\sigma}(\tilde{x}) = \sigma(x)/s_y = T_y(\sigma(x)) - T_y(0); \quad (8)$$

applying the full affine map here would be incorrect, silently shifting the band by the normalisation offset μ_y/s_y . The second identity in (8) is how the implementation computes the scale-only transform: it only ever holds the normaliser as an opaque callable, and $T_y(\sigma) - T_y(0)$ extracts the scale without access to the normaliser’s parameters, for any affine T_y .

Because the normalisers change between refits, the mean and kernel cannot capture T_x, T_y once at construction time; the implementation instead binds them by *late-bound closures* that read the current normaliser at call time (Section 3.3).

2.5 Cost

The proxy is evaluated at every posterior query, which during acquisition optimisation (here: dual annealing with sequential proximity-punished batch selection) means thousands of calls per suggested batch. Microsecond- to millisecond-scale proxies are negligible; second-scale proxies (semi-empirical methods) dominate the suggestion step and benefit from memoisation, which we leave as future work. Against an objective costing days to months per evaluation, even the un-memoised cost is irrelevant.

3 Implementation in veropt

The method is implemented in the open-source Bayesian optimisation package **veropt** (Stoustrup et al., 2026), which was built for tuning the VEROS ocean simulator (Häfner et al., 2018) and targets exactly the expensive-objective regime considered here. This section documents the implementation at the level of the code, both as a map for readers of the source and as a template for adding the construction to other GP stacks. Listings are taken verbatim from the package, with type annotations, docstrings, and defensive assertions elided and comments occasionally abridged or tagged with equation numbers; the named source files are authoritative. Table 1 maps each equation to the code that realises it.

3.1 The host: veropt’s model stack

To read the changes, four facts about the package suffice. (i) **BayesianOptimiser** (`optimiser.py`) drives the loop and holds all data in physical (“real”) units; it owns the normalisers and refits them as configured. (ii) The surrogate is a **GPyTorchFullModel** (`model.py`): a list of *independent* single-objective GPs (**GPyTorchSingleModel**), one per objective. Each single model holds a `mean_module` (by default `gpytorch.means.ConstantMean()`) and a kernel; the six kernels offered by the package (Matérn, double Matérn, rational quadratic, a rational-quadratic–Matérn product, spectral mixture, spectral delta) are subclasses configuring that pair. (iii) Training

Table 1: Where each piece of the mathematics lives in the code. All paths are relative to `veropt/optimiser/`.

Quantity	Equation	Code
proxy g	—	user subclass of <code>ProxyMeanFunction</code>
whitened mean $\tilde{m}$	(7)	<code>ProxyMean.forward</code>
band $\sigma(x)$, whitened	(3), (8)	<code>ProxyScaledKernel._sigma</code>
scale-only transform	(8)	<code>scale_objectives_to_normalised</code>
correlation ρ	(4)	<code>ProxyScaledKernel._scaling</code>
full kernel k	(2)	<code>ProxyScaledKernel.forward</code>
amplitude cap $c \in (c_{\min}, 1]$	(5)	Interval on <code>raw_amplitude_factor</code>
late-bound T_x^{-1}, T_y	§2.4	closure getters in <code>optimiser.py</code>

builds a fresh `ExactGP` around the current (whitened) data and runs Adam on the exact marginal log-likelihood; the mean and kernel *modules* persist across steps, so hyperparameters carry over. (iv) Everything the model sees — training data, acquisition queries — is in whitened units.

The feature therefore decomposes into: a new self-contained module supplying the mean and the kernel wrapper (§3.2); small changes to the model classes to install them and keep serialisation working (§3.3); plumbing that hands the current normalisers to the model at the right moment (§3.3); and the user-facing configuration surface (§3.4). The acquisition functions, the acquisition optimisers, the normalisers, the training loop, and the plotting code are untouched.

3.2 The new module: `proxy_prior.py`

The user contract: `ProxyMeanFunction`. The user supplies the proxy by subclassing `ProxyMeanFunction`, an abstract savable class mirroring the package’s objective classes: physical units in, physical units out. The base class validates shapes and finiteness around the user’s `_run` method — input `[n_points, n_variables]`, output `[n_points]` — so a defective proxy fails loudly at the boundary rather than deep inside a Cholesky factorisation. A concrete subclass appears in the example of Section 4.3.

The prior mean: `ProxyMean`. A `gpytorch.means.Mean` that implements (7). It holds the user’s proxy and two initially unset normalisation closures, installed later by `update_normalisation_functions`:

```
class ProxyMean(gpytorch.means.Mean):

    def __init__(self, mean_function):
        super().__init__()
        self.mean_function = mean_function
        self._unnormaliser_variables = None    # x-tilde -> x
        self._normaliser_objectives = None    # y -> y-tilde

    def evaluate_proxy_real_units(self, variable_values):
        # g(T_x^{-1}(x)): unnormalise the query points, evaluate
        # the proxy in real units

        # botorch queries the mean with batched points
        # [..., n_points, n_variables]; the proxy contract is a plain
        # 2D matrix, so flatten and restore around the call
        output_shape = variable_values.shape[:-1]
        flattened_values = variable_values.reshape(
```

```

        -1, variable_values.shape[-1])

    # Evaluated without gradients: safe with the current
    # derivative-free acquisition optimisers
    with torch.no_grad():
        proxy_values = self.mean_function(
            variable_values=self._unnormaliser_variables(
                flattened_values)
        )

    return proxy_values.reshape(output_shape)

def forward(self, x):
    # m(x) = T_y(g(T_x^{-1}(x))): the prior mean is the proxy,
    # mapped through the current normalisations [eq. (7)]
    proxy_values_real_units = \
        self.evaluate_proxy_real_units(variable_values=x)
    with torch.no_grad():
        return self._normaliser_objectives(proxy_values_real_units)

```

`forward` is (7) read left to right: unwhiten the query points, evaluate the proxy, whiten the result. The reshape bookkeeping exists because `botorch` evaluates means on batched inputs $[\dots, n, d]$ during acquisition optimisation, while the user contract is a plain $[n, d]$ matrix; the batch dimensions are flattened for the proxy call and restored afterwards. The same class provides the scale-only transform of (8), used by the kernel below:

```

def scale_objectives_to_normalised(self, objective_values):
    # Transforms a *deviation* (rather than a value): applies only
    # the scale of the normalisation, not the shift.
    with torch.no_grad():
        transformed_values = \
            self._normaliser_objectives(objective_values)
        transformed_zeros = \
            self._normaliser_objectives(
                torch.zeros_like(objective_values))
    return transformed_values - transformed_zeros

```

The prior covariance: ProxyScaledKernel. A `gpytorch.kernels.Kernel` that wraps *any* of the package’s kernels as the base kernel k_b and implements (2) with the diagonal normalisation (4) built in. Writing the kernel as

$$k(x, x') = \underbrace{\frac{c \sigma(x)}{\sqrt{k_b(x, x)}}}_{a(x)} \underbrace{\frac{c \sigma(x')}{\sqrt{k_b(x', x')}}}_{a(x')} k_b(x, x'),$$

the `forward` is nothing but this equation — the outer product of the vector a with itself, times the base kernel matrix — with everything else delegated to named helpers:

```

class ProxyScaledKernel(gpytorch.kernels.Kernel):

    has_lengthscale = False

    def forward(self, x1, x2, diag=False, **params):
        # k(x, x') = a(x) a(x') k_b(x, x') [eq. (2)]
        scaling_1 = self._scaling(x1, **params)
        scaling_2 = self._scaling(x2, **params)

```

```

base_values = self._evaluate_base_kernel(
    x1, x2, diag=diag, **params)

if diag:
    return scaling_1 * scaling_2 * base_values
else:
    return scaling_1.unsqueeze(-1) \
        * scaling_2.unsqueeze(-2) * base_values

```

The scaling $a(x)$ divides by the base kernel’s own diagonal — the normalisation (4) that keeps the band calibrated for base kernels with $k_b(x, x) \neq 1$:

```

def _scaling(self, variable_values, **params):
    # a(x) = c * sigma(x) / sqrt(k_b(x, x)) [eq. (4)]
    # - dividing by the base kernel’s diagonal makes it a
    # correlation kernel, so the prior variance is exactly
    # c^2 sigma(x)^2 even for base kernels with
    # k_b(x, x) != 1 (e.g. kernel sums, spectral mixtures)
    base_diagonal = self._evaluate_base_kernel(
        variable_values, variable_values, diag=True, **params)
    .clamp(min=1e-12).sqrt()

    return self.amplitude_factor * self._sigma(variable_values) \
        / base_diagonal

```

The whitened band $\tilde{\sigma}$ of (3)/(8) is computed by `_sigma`, with the relative/absolute bound choice and the numerical floor kept apart in `_deviation_bound_real_units`:

```

def _sigma(self, variable_values):
    # sigma(x) = epsilon(x) / kappa [eq. (3)]
    sigma_real_units = \
        self._deviation_bound_real_units(variable_values) \
        / self.settings.bound_in_n_sigmas

    # sigma-tilde = T_y(sigma) - T_y(0) [eq. (8)]
    return self.proxy_mean.scale_objectives_to_normalised(
        objective_values=sigma_real_units)

def _deviation_bound_real_units(self, variable_values):
    # epsilon(x): the promised pointwise deviation band,
    # in real objective units [eq. (1)]
    if self.settings.bound_type == 'relative':
        proxy_values = \
            self.proxy_mean.evaluate_proxy_real_units(
                variable_values=variable_values)
        return (
            self.settings.bound_value * proxy_values.abs()
        ).clamp(min=self.settings.amplitude_floor)

    elif self.settings.bound_type == 'absolute':
        return torch.full(
            size=variable_values.shape[:-1],
            fill_value=self.settings.bound_value)

```

Note that the band is computed entirely in physical units until the last line of `_sigma` — the guarantee, the floor, and κ all have physical meaning — and only then is the scale-only whitening applied. `ProxyScaledKernel` declares `has_lengthscales = False`: it owns no `lengthscale` and

no outputscale of its own, so the only hyperparameters under the optimiser’s control are the base kernel’s (reached through the child module) and the amplitude factor c .

The amplitude cap. The constraint (5) uses gpytorch’s standard reparametrisation machinery: the raw parameter is unconstrained, and an `Interval` constraint maps it through a scaled sigmoid into $(c_{\min}, 1]$. Gradient training can therefore push c towards the cap but never across it.

```
if settings.train_amplitude_factor:
    self.register_parameter(
        name='raw_amplitude_factor',
        parameter=torch.nn.Parameter(torch.zeros(())))

    # The deviation bound is an upper bound: the amplitude may
    # shrink with training but can never exceed the bound.
    self.register_constraint(
        param_name='raw_amplitude_factor',
        constraint=Interval(
            lower_bound=settings.amplitude_factor_lower_bound,
            upper_bound=1.0))

    # The sigmoidal transform is infinite at the boundary, so
    # initialise just inside the cap.
    self.amplitude_factor = torch.tensor(0.99)
```

With `train_amplitude_factor` off, the parameter is never registered and the `amplitude_factor` property returns the constant 1: the literal worst-case prior. The initialisation at 0.99 rather than 1.0 is forced by the reparametrisation: the inverse sigmoid is infinite at the boundary.

Settings. All options travel in a small dataclass, `ProxyPriorSettings`, validated from the user-facing typed dictionary (Table 2 in Section 4); `ProxyPrior` bundles the user’s mean function with these settings and is the unit that the model classes install and serialise.

3.3 Wiring into the existing optimiser core

Four existing files were changed. The changes are small and local, which is the point of the design: the prior is confined to the mean and kernel modules, exactly where GP theory says it belongs.

Installing the prior (model.py). A `GPyTorchSingleModel` accepts a proxy prior only *before* it has seen data — swapping the prior under a trained model would silently invalidate its hyperparameters:

```
def set_proxy_prior(self, proxy_prior):
    assert self.model_with_data is None, \
        "The proxy prior must be set before the model is given data"

    self.proxy_prior = proxy_prior

    proxy_mean = ProxyMean(
        mean_function=proxy_prior.mean_function)

    self.mean_module = proxy_mean          # replaces ConstantMean
```

```

self.kernel = ProxyScaledKernel(      # wraps the chosen kernel
    base_kernel=self.kernel,
    proxy_mean=proxy_mean,
    settings=proxy_prior.settings)

```

Because the base kernel is registered as a child module of the wrapper, its hyperparameters remain visible to the training loop and appear in the state dictionary under `covar_module.base_kernel.*` — nothing else in the training machinery needs to know the wrapper exists.

Three consequences did need attention. First, kernel-specific code (lengthscale constraints and getters in `kernels.py`) reaches for `model_with_data.covar_module` expecting the base kernel; a new property transparently unwraps:

```

@property
def data_model_base_covar_module(self):
    covar_module = self.model_with_data.covar_module
    if isinstance(covar_module, ProxyScaledKernel):
        return covar_module.base_kernel
    else:
        return covar_module

```

and the six kernel classes were mechanically switched to it. Second, `ProxyMean` — unlike the `ConstantMean` it replaces — has no trainable parameters, and `torch.optim.Adam` raises on an empty parameter group; the parameter groups handed to the optimiser are now filtered through a helper, `filter_out_empty_parameter_groups`. Third, serialisation: the saved state gains a `proxy_prior` entry (the user proxy’s name and constructor state plus the settings), and `from_saved_state` re-applies `set_proxy_prior` *before* loading the state dictionary, so that the saved keys (`covar_module.base_kernel.*`, `raw_amplitude_factor`) find their modules. Files saved before the feature existed load unchanged.

Handing over the normalisers (`optimiser.py`, `prediction.py`). Section 2.4 requires the mean and kernel to see the *current* T_x^{-1} and T_y , across refits. The optimiser exposes them as closures over `self`, so a single installation stays fresh forever:

```

def get_unnormaliser_function_variables(self):
    def unnormalise_variables(variable_values):
        if self.return_normalised_data:
            variable_values = self._normaliser_variables \
                .inverse_transform(variable_values)
        return variable_values
    return unnormalise_variables

```

The closure reads `self._normaliser_variables` at *call* time, so a normaliser refit is picked up automatically; and if normalisation is disabled entirely (`normalise=False`), the guard makes the closure the identity and the whole construction transparently operates in physical units. The pair of closures is injected at the top of the optimiser’s predictor update — before training, because training is the first thing that evaluates the mean:

```

def _update_predictor(self, train=True):
    # Must happen before training so the proxy prior mean can map
    # between unit systems. The closures read the current
    # normalisers at call time, so they survive refits.
    self.predictor.update_model_normalisation_functions(
        unnormaliser_variables=
            self.get_unnormaliser_function_variables(),
        normaliser_objectives=

```

```

        self.get_normaliser_function_objectives()

    self.predictor.update_with_new_data(
        variable_values=self.evaluated_variable_values.tensor,
        objective_values=self.evaluated_objective_values.tensor,
        train=train)
    ...

```

`update_model_normalisation_functions` is a no-op on the `Predictor` base class and is forwarded by `BotorchPredictor` to the model, which forwards it to any `ProxyMean` present — models without a proxy prior are untouched by the entire code path. In the multi-objective case each single-objective GP receives an adapter that restricts the $[\dots, n_{\text{obj}}]$ objective normaliser to its own column (`make_single_column_objective_normaliser`), since each GP whitens against its own objective’s statistics.

3.4 The user-facing API (`constructors.py`)

The package’s constructor, `bayesian_optimiser(...)`, configures the model through a typed dictionary; it gains two keys, `proxy_prior` and `proxy_prior_settings`. A single proxy is broadcast to all objectives; a list supplies one entry per objective, with `None` leaving that objective at the uninformative default. Validation is eager and specific: a settings dictionary without a proxy, a list of the wrong length, a proxy whose `n_variables` disagrees with the objective’s, or an unknown settings key each fail at construction time with a message naming the offending objective. The mechanism composes with everything else the constructor does, because `set_proxy_prior` is applied to the freshly built single-objective models before any data reaches them.

The implementation is exercised by 23 dedicated tests (`tests/test_proxy_prior.py`) covering the calibration identity $k(x, x) = c^2 \tilde{\sigma}(x)^2$ for all six kernels (including the non-correlation ones), the scale-only whitening, the amplitude cap under training, multi-objective mixing, serialisation round-trips, and constructor validation — a useful specification of intended behaviour for readers extending the feature.

4 Using proxy-informed priors

This section is the user guide: what a proxy must satisfy, what the options mean, and a complete example to copy from.

4.1 Writing a proxy

The proxy is a subclass of `ProxyMeanFunction` (importable directly from `veropt`) with three obligations:

1. **A unique name class attribute.** It identifies the class in save files, exactly as for user-defined objectives.
2. **A `_run` method in physical units, vectorised over points.** Input is a torch tensor of shape $[\text{n_points}, \text{n_variables}]$ holding real variable values; output is the proxy objective per point, shape $[\text{n_points}]$ (a column $[\text{n_points}, 1]$ is accepted and flattened). Since `veropt` maximises, a proxy for an energy to be minimised returns the negated energy.
3. **Tolerance of boundary round-off.** Whitening and unwhitening can hand the proxy points a floating-point epsilon outside the variable bounds; a proxy with a strictly validated domain should clamp its input (one line, visible in the example below).

Table 2: `proxy_prior_settings` keys. “Physical units” means the objective’s own units, before any normalisation.

Key	Default	Meaning
<code>bound_value</code>	required	r (relative) or ε_0 , physical units (absolute)
<code>bound_type</code>	‘relative’	reading of <code>bound_value</code> : ‘relative’ or ‘absolute’
<code>bound_in_n_sigmas</code>	2.0	κ in (3): band = κ prior std. deviations
<code>train_amplitude_factor</code>	True	train c by marginal likelihood; else $c \equiv 1$
<code>amplitude_factor_lower_bound</code>	0.1	$c_{\min}$ in (5)
<code>amplitude_floor</code>	0.0	$\varepsilon_{\text{floor}}$, physical units; keeps $\sigma > 0$ where $g \approx 0$

Speed matters: the proxy is called thousands of times per suggestion step inside the acquisition optimiser, so a call (over a whole batch of points) should cost well under a second. It is evaluated under `torch.no_grad()`, so it may use non-differentiable operations freely.

4.2 Configuring the optimiser

The proxy and its guarantee enter through the model choice dictionary:

```
from veropt import bayesian_optimiser

optimiser = bayesian_optimiser(
    n_initial_points=8,
    n_bayesian_points=32,
    n_evaluations_per_step=4,
    objective=expensive_objective,
    model={
        'kernels': 'matern',
        'proxy_prior': my_proxy,
        'proxy_prior_settings': {'bound_value': 0.01},
    },
    n_points_before_fitting=4,
)
```

Two of these lines deserve comment. `bound_value` is the guarantee and the only required setting: here, “the true objective is within 1% of the proxy”. And `n_points_before_fitting` is lowered to a single batch: by default the model is first fitted only towards the end of the random initial phase, which is the right economy for an uninformative prior but wastes the head start an informative one provides — with a proxy prior the model is worth fitting (and its amplitude factor worth training) as soon as the normalisers can be estimated. Table 2 lists all settings.

For multi-objective problems, supply one proxy (or `None`) and one settings dictionary (or `None`) per objective:

```
model={
    'kernels': 'matern',
    'proxy_prior': [energy_proxy, None],
    'proxy_prior_settings': [{'bound_value': 0.01}, None],
},
```

Saving and loading work as for user-defined objectives: `save_to_json` stores the proxy’s name, constructor state and settings, and `load_optimiser_from_state` re-installs the prior — provided the proxy subclass has been imported in the loading process, which is the same rule the package already imposes on objective classes.

4.3 A complete example

The listing below is the complete, runnable script shipped as `examples/example_proxy_informed_prior.py`. The “expensive” objective is the six-dimensional Hartmann function (a standard multimodal BO benchmark, maximised, $f^* = 3.3224$); the proxy is the same function with a 1% multiplicative sinusoidal distortion, so the relative guarantee `bound_value = 0.01` holds *exactly* by construction while the distortion shifts the local geometry — the honest analogue of a semi-empirical surface against a coupled-cluster one.

```
from typing import Literal

import botorch
import torch

from veropt import ProxyMeanFunction, bayesian_optimiser
from veropt.optimiser.practice_objectives import Hartmann

class DistortedHartmannProxy(ProxyMeanFunction):

    name = 'distorted_hartmann_proxy'

    def __init__(self, n_variables: int) -> None:
        super().__init__(n_variables=n_variables)
        self.function = botorch.test_functions.Hartmann(
            dim=n_variables,
            negate=True
        )

    def _run(self, variable_values: torch.Tensor) -> torch.Tensor:
        # The proxy can receive values a floating point rounding
        # error outside the bounds, so functions with a strictly
        # validated domain (like botorch’s) should clamp
        variable_values = variable_values.clamp(0.0, 1.0)

        true_values = self.function(variable_values)

        return true_values * (
            1.0 + 0.01 * torch.sin(
                6.0 * torch.pi * variable_values[:, 0])
        )

n_variables: Literal[6] = 6
n_evaluations_per_step = 4

objective = Hartmann(n_variables=n_variables)

proxy = DistortedHartmannProxy(n_variables=n_variables)

optimiser = bayesian_optimiser(
    n_initial_points=8,
```

```

n_bayesian_points=32,
n_evaluations_per_step=n_evaluations_per_step,
objective=objective,
model={
    'kernels': 'matern',
    'proxy_prior': proxy,
    'proxy_prior_settings': {
        # the objective is within 1% of the proxy...
        'bound_value': 0.01,
        # ...read as two standard deviations of the prior
        'bound_in_n_sigmas': 2.0,
        # keeps the prior variance away from zero where
        # the proxy is near zero
        'amplitude_floor': 0.01
    }
},
n_points_before_fitting=n_evaluations_per_step
)

for step in range(4):
    optimiser.run_optimisation_step()

best_objective_value = float(
    optimiser.evaluated_objectives_real_units.max()
)

print(
    f"\nBest objective value found: {best_objective_value:.4f} "
    f"(the optimum is 3.3224)"
)

```

To run it from a checkout of the repository (Python > 3.13):

```

pip install .
python examples/example_proxy_informed_prior.py

```

The four optimisation steps evaluate 16 points: two batches of 4 random initial points, then two model-suggested batches. The run recorded for this paper took four minutes on a single CPU core (the cost is dominated by the acquisition optimiser, not the GP) and finished with

```
Best objective value found: 3.3148 (the optimum is 3.3224)
```

a 0.2% optimality gap after 16 evaluations of the “expensive” function, reached with the first model-suggested batch rather than gradually. Runs are not bit-reproducible (the acquisition optimiser draws from an unseeded global random state), but this behaviour is stable across repetitions; Section 5 quantifies it over five paired repetitions against a flat prior.

4.4 Adapting the example to your own problem

The distance from the example to a real application is deliberately short:

1. **Wrap your fast model.** Replace `DistortedHartmannProxy._run` with your surrogate — a harmonic energy, a semi-empirical method, a coarse-grid simulation — returning the (negated, if minimising) objective per input row, in physical units.
2. **State your guarantee honestly.** A percentage bound (“within 1%”) is `bound_type='relative'` with `bound_value=0.01`; a uniform error bar (“within 1 kcal/mol”) is `bound_type='absolute'` with `bound_value` in the objective’s units. The

Table 3: Best objective value found (mean $\pm$ standard deviation over five seeds) after n evaluations of the Hartmann-6 objective ($f^* = 3.3224$). Both methods share seeds, budget, kernel, and acquisition machinery; they differ only in the prior.

n	Flat prior	Proxy-informed prior
8 (random)	1.117 ± 0.644	1.117 ± 0.644
12	1.719 ± 0.451	3.3152 ± 0.0010
16	2.108 ± 0.387	3.3177 ± 0.0041
20	2.562 ± 0.247	3.3177 ± 0.0041
24	2.731 ± 0.276	3.3193 ± 0.0041

bound is a promise the optimiser will exploit: if the truth escapes the band, the model can become confidently wrong and the acquisition may never look where the truth is better than the proxy suggests. If any expensive evaluations exist already, check them against the bound before promising it.

3. **Guard the zero crossings.** Under a relative bound, set `amplitude_floor` to a small value in physical units if the objective changes sign or passes near zero anywhere in the domain.
4. **Start fitting early.** Set `n_points_before_fitting` equal to `n_evaluations_per_step`.
5. **Leave the rest alone.** Kernels, acquisition, batching and normalisation all compose with the prior unchanged; multi-objective problems take a list of proxies with `None` for uninformed objectives.

5 Experiments

We benchmark the example configuration of Section 4.3 at a larger budget, against an identical optimiser with the default flat prior. The objective is the six-dimensional Hartmann function (maximised, optimum $f^* = 3.3224$) and the proxy is the multiplicative harmonic distortion

$$g(x) = f(x) (1 + 0.01 \sin(6\pi x_1)), \quad (9)$$

so that the relative bound (1) with $r = 0.01$ holds exactly, while the distortion shifts the local geometry of the optimum. Both methods use a Matérn-5/2 kernel with ARD, batches of 4, 8 initial random points followed by 16 Bayesian-phase points (a total budget of 24 evaluations), and `veropt`’s defaults otherwise. The proxy-informed runs use $\kappa = 2$, a trainable amplitude $c \in (0.1, 1]$, and an absolute band floor of $\varepsilon_{\text{floor}} = 0.01$. Each configuration is repeated five times with paired initial points (the torch seed is shared between arms; the acquisition optimiser retains its own stochasticity).

Figure 1 and Table 3 report the best objective value found as a function of the number of evaluations n , over five paired repetitions (the 8 initial random points are identical between the two arms). The effect of the prior is immediate: the *first* model-suggested point already lands at the global optimum, and after one Bayesian batch ($n = 12$) every proxy-informed run sits within 0.008 of the optimum — a relative optimality gap of 0.2%, i.e. at the scale of the proxy’s own error band — while the across-repetition spread collapses from ± 0.45 to ± 0.001 . The flat prior, by contrast, is still climbing at the end of the budget and terminates 0.59 below the optimum on average (an 18% gap) with nearly seventy times the spread. Reading the comparison in terms of evaluations rather than attained value: the proxy-informed optimiser needs 12 evaluations to surpass the level the flat prior has not reached after 24. For an objective costing months per

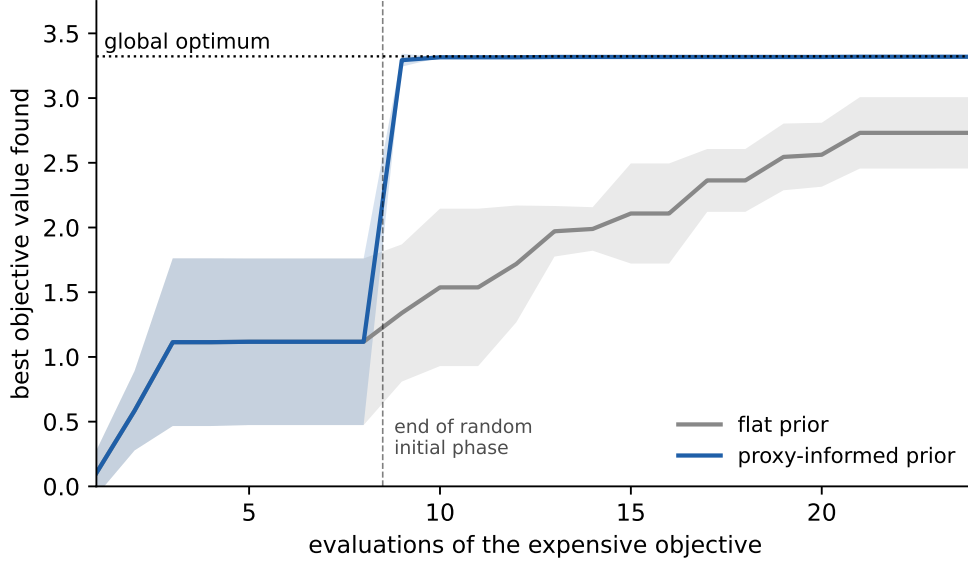


Figure 1: Best objective value found versus number of expensive evaluations on Hartmann-6 (mean over five paired repetitions; shaded bands are one standard deviation). The two arms share their 8 random initial points, so the curves coincide up to the dashed line. The first point suggested by the proxy-informed model already lands at the global optimum.

evaluation, this is the difference between one and several years of computation. The trained amplitude factor is doing its intended work: across repetitions it settled at $c \in [0.69, 0.89]$, i.e. the marginal likelihood tightened the band 10–30% below the worst-case bound without ever being allowed to exceed it. We note the benchmark is deliberately favourable — the proxy is everywhere within 1% of the truth and the distortion is smooth — but this is precisely the regime the method is designed for, and the guarantee that makes it favourable is one the user must certify, not one the method assumes silently.

6 Limitations and future work

Soft bound. The hard guarantee (1) is encoded as a κ -sigma Gaussian band; at $\kappa = 2$, roughly 5% of prior mass violates the bound pointwise. Truncated or constrained GPs would encode the bound exactly at considerably greater inferential cost.

Calibration of the residual smoothness. The bound fixes the *amplitude* of the residual but not its *lengthscale*; the latter is learned from data as usual. A proxy whose error oscillates faster than the learned lengthscale will be modelled overconfidently between evaluations.

Zero-data suggestion. With an informative prior the surrogate is meaningful before any expensive evaluation, so in principle the initial random phase could be replaced by acquisition over the prior alone. The current implementation still requires one batch of evaluations before the model is activated (its normalisers need at least two points); removing this restriction is mechanical but not yet done.

Gradients and memoisation. The proxy is currently evaluated without gradients — harmless under derivative-free acquisition optimisation, but a gradient-based acquisition optimiser would require differentiating through the prior mean. Memoisation of proxy calls would make second-scale proxies practical.

7 Conclusion

When an expensive objective comes with a cheap surrogate and a quantitative pointwise error guarantee — the everyday situation in computational chemistry and many other simulation sciences — that guarantee can be installed directly into the Gaussian process prior of a Bayesian optimiser: proxy as prior mean, error band as (heteroscedastic) prior standard deviation, bound as a hard cap on a trained amplitude. The construction is exact in its bookkeeping (valid PSD covariance, correct behaviour under data whitening), composes with standard kernels and acquisition machinery, and is implemented in **veropt** in one self-contained module plus a handful of local changes, documented here alongside a runnable example. In benchmark experiments it converts a months-per-point optimisation that would otherwise spend its budget rediscovering known structure into one that starts at the proxy and spends every evaluation on the residual.

Acknowledgements

The proxy-informed prior is implemented in **veropt**, developed by Aster Stoustrup and collaborators; this work builds directly on that foundation.

References

- R. Astudillo and P. I. Frazier. Thinking inside the box: A tutorial on grey-box Bayesian optimization. In *Proceedings of the 2021 Winter Simulation Conference*, 2021.
- A. I. J. Forrester, A. Söbester, and A. J. Keane. Multi-fidelity optimization via surrogate modelling. *Proceedings of the Royal Society A*, 463:3251–3269, 2007.
- P. I. Frazier. A tutorial on Bayesian optimization. *arXiv:1807.02811*, 2018.
- J. R. Gardner, G. Pleiss, D. Bindel, K. Q. Weinberger, and A. G. Wilson. GPyTorch: Blackbox matrix-matrix Gaussian process inference with GPU acceleration. In *Advances in Neural Information Processing Systems*, 2018.
- D. Häfner, R. L. Jacobsen, C. Eden, M. R. B. Kristensen, M. Jochum, R. Nuterman, and B. Vinter. Veros v0.1 – a fast and versatile ocean simulator in pure Python. *Geoscientific Model Development*, 11:3299–3312, 2018.
- M. C. Kennedy and A. O’Hagan. Predicting the output from a complex computer code when fast approximations are available. *Biometrika*, 87(1):1–13, 2000.
- B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016.
- A. Stoustrup et al. **veropt**: the versatile optimiser. <https://github.com/aster-stoustrup/veropt>, 2026.